RAPPORT DE STAGE :

Sujet : Développement d'une application web de recherche des codes postaux

Réalisé par :

Malcolm BERTAINA (L3T7 Informatique)

Sous la direction de :

Sabrina VEROLLE (Tutrice OPT-NC)
Jimmy AVAE (Encadrant technique OPT-NC)
Loïc SALMON (Tuteur UNC)

Formation : Licence Informatique

Année : 2024

Table des matières :

1. Remerciements **1**
2. Introduction **1**
2.1. Recherches 1
2.2. Sujet 1
2.3. Objectifs 1
3. Présentation **2**
3.1. L'OPT-NC 2
3.2. La DSI 2
3.3. Le Postal 3
4. Organisation **4**
5. Architecture **5**
6. Développement Backend (API) **5**
6.1. Choix des outils 5
6.2. Alive 6
6.3. Recherche par localité 6
6.4. Recherche par numéro de boîte postale 8
6.5. Swagger 10
6.6. Conteneurisation 10
7. Développement Frontend (Interface) **10**
7.1. Choix des outils 10
7.2. Maquettage 11
7.3. Recherche par localité 11
7.4. Recherche par numéro de boîte postale 13
7.5. Intégration et déploiement continus (CI/CD) 15
8. Ateliers **15**
8.1. Définir les cas d'utilisation de l'API 15
8.2. Présentation de mon travail auprès du métier 16
8.3. Présentation finale auprès du métier 16
9. Problèmes rencontrés **17**
9.1. Difficulté à trouver un stage 17
9.2. Compréhension de la base de données SIBP 17
9.3. Gestion des recherches par code postal 17
9.4. Gestion des intervalles de boîtes postales 17
10. Conclusion **18**
11. Glossaire **19**
12. Références **21**
13. Annexes **22**

1. Remerciements

Je tiens tout d'abord à remercier l'**OPT-NC** et plus particulièrement **Sabrina VEROLLE**, ma tutrice, pour m'avoir proposé ce sujet de stage et pour m'avoir fait confiance en m'accueillant au sein de sa section.

Je remercie également **Jimmy AVAE**, mon encadrant technique, pour son soutien. Le fait de m'avoir assigné des tickets adaptés à mes compétences tout en me faisant voir de nouvelles notions m'a permis de monter en compétences et de mieux appréhender les aspects techniques du projet.

Je voudrais ensuite remercier **David BROUSTE**, qui, bien que n'ayant pas pour mission de m'encadrer, n'a pas hésité à me suggérer des améliorations sur l'Expérience Utilisateur (UX) de mon application, ce qui a été d'une grande aide.

Enfin, un grand merci à l'ensemble de l'équipe du SI Postal, qui m'a réservé un accueil bienveillant et qui n'a pas hésité à me prodiguer de nombreux conseils utiles.

2. Introduction

2.1. Recherches

Dans le cadre de ma dernière année de licence en Informatique, j'ai effectué un stage obligatoire de 2 mois, du 4 novembre 2024 au 31 décembre 2024, au sein de **l'Office des Postes et Télécommunications de Nouvelle-Calédonie (OPT-NC)**.

J'ai commencé mes recherches de stage assez tôt, dès le mois d'avril. Cependant, des émeutes et des destructions ont suivi, impactant gravement de nombreuses entreprises en Nouvelle-Calédonie, ce qui a rendu mes recherches particulièrement difficiles.

Malgré ces obstacles, après de nombreuses candidatures spontanées et relances, j'ai finalement reçu une proposition de stage de la part de l'OPT-NC en septembre, ce qui m'a permis de commencer mon stage le 4 novembre 2024.

2.2. Sujet

Le sujet de mon stage consistait à développer une **application web de recherche des codes postaux**, destinée à simplifier l'accès aux informations liées aux codes postaux. Actuellement, la recherche des codes postaux s'avère peu pratique pour les utilisateurs, car ces informations sont présentées sous la forme d'un simple tableau disponible sur une page du site web de l'OPT-NC, sans réelle interactivité.

2.3. Objectifs

L'objectif de cette application est de permettre aux utilisateurs de trouver facilement les informations associées à un **code postal** à partir d'une localité ou d'un numéro de boîte postale. Ce projet vise à moderniser et à améliorer l'expérience utilisateur en automatisant une tâche jusqu'ici fastidieuse.

Voici à quoi ressemble un extrait du tableau des codes postaux :

NOUMEA BOITES POSTALES			
Agence OPT-NC correspondante (pour info)	Libellé de l'adresse Numéro de BP	Code Cedex	Libellé Cedex
AG PRINCIPALE	1 à 1900	98845	NOUMEA CEDEX
AG PRINCIPALE	1901 à 4200	98846	NOUMEA CEDEX
AG PRINCIPALE	4201 à 5100	98847	NOUMEA CEDEX

3. Présentation

3.1. L'OPT-NC

Établissement public à caractère industriel et commercial (EPIC), l'**OPT-NC** remplit une mission essentielle d'aménagement et de développement du territoire, au service des populations, des entreprises et des collectivités.

Avec un effectif de plus de **1 000 agents** et une diversité de missions au cœur du quotidien des Calédoniens, l'OPT-NC est l'un des principaux établissements publics du pays. Son réseau, présent sur l'ensemble de la Grande Terre et des Îles, permet de desservir même les zones les plus isolées.

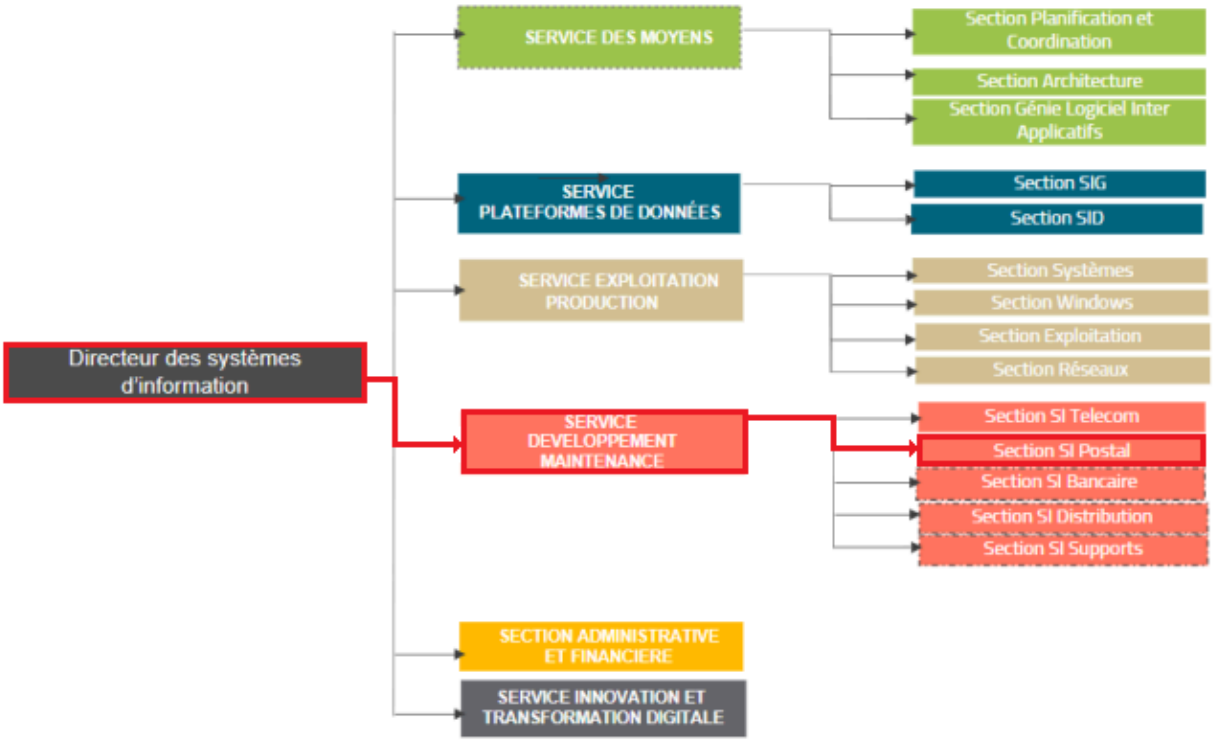
L'OPT-NC est présent sur l'ensemble de la Nouvelle-Calédonie, auprès de ses clients particuliers et professionnels, à travers ses quatre métiers principaux :

- **les télécommunications,**
- **le courrier-colis,**
- **les services financiers,**
- **et le réseau de distribution.**

3.2. La DSI

Durant mon stage, j'ai eu l'opportunité de découvrir l'ensemble des différentes sections de la **Direction des Systèmes d'Information (DSI)** et de rencontrer ses membres. Cette visite m'a permis de mieux comprendre l'organisation interne de la DSI et la manière dont les équipes sont réparties pour gérer les multiples projets technologiques de l'OPT-NC.

Ci-dessous, l'organigramme de la DSI illustre cette organisation.



3.3. Le Postal

En quelques chiffres, la direction postal représente un chiffre d'affaires (CA) d'environ 2,6 milliards de francs en 2023, soit 11 % du CA total de l'OPT-NC pour cette même année.

Les **missions principales** de la direction postal incluent :

- Assurer l'acheminement et la distribution des lettres, paquets et colis
- Proposer des offres de produits et services adaptés aux besoins et usages des clients
- Mettre en œuvre un programme philatélique annuel pour les passionnés de timbres

Durant ce stage, j'ai donc évolué au sein du **Système d'Information Postal (SI Postal)**, un département qui gère un grand nombre d'applications essentielles au bon fonctionnement des services postaux. Parmi ces applications, on retrouve **le Système d'Information des Boîtes Postales (SIBP)**, **le Portail des Services Postaux (PSP)**, **le Suivi Informatisé Des Objets (SIDO)**, **SIDO Mobile, les Machines à Affranchir (MAF) et les Ordres de Réexpédition (OR).**

À l'exception de PSP, toutes ces applications sont destinées à un usage interne et ne sont pas accessibles au grand public.

Bien qu'il existe d'autres applications au sein du SI Postal, j'ai principalement travaillé sur SIBP, car j'ai dû reprendre sa base de données pour fournir les informations requises par l'API. Actuellement, la DSI de l'OPT est en cours de migration vers GitHub. Cependant, le projet repose sur SIBP hébergée dans GitLab, où l'API sera intégrée à terme.

Application	Description
SIBP	Outil de gestion des boîtes postales, qui permet de gérer les clients, les contrats, les boîtes postales, etc.
PSP	Portail destiné aux clients pour le suivi de leurs colis, le paiement des frais de douane, et d'autres services. Il est accessible à l'adresse https://monespace.opt.nc
SIDO	Outil permettant de suivre les objets postaux (colis, courrier recommandé) une fois arrivés en Nouvelle-Calédonie, au CTP de Ducos. Les agents du CTP scannent le code-barres du courrier pour qu'il soit enregistré dans SIDO et dans PSP.
SIDO Mobile	Utilisée par les facteurs de l'OPT-NC, cette application permet la distribution et la gestion des objets postaux. Elle permet également de faire signer la réception des colis par les clients, et d'attribuer un statut à chaque livraison (distribué, destinataire absent, etc.).
MAF	Application back-office utilisée par le service facturation de l'OPT pour gérer les consommations en affranchissements de clients professionnels équipés de machines à affranchir.
OR	Outil de gestion des ordres de réexpédition et garde de courriers. Utilisé en agence OPT.

4. Organisation

L'organisation de mon travail s'est principalement articulée autour des tickets créés sur la plateforme **Jira**, qui m'étaient assignés par mes encadrants. Les tickets étaient organisés sous la forme d'un **tableau Kanban**, une méthode visuelle qui permet de suivre l'évolution des tâches de manière claire et structurée.

Le tableau Kanban était divisé en plusieurs colonnes correspondant aux différentes étapes du processus de travail, comme :

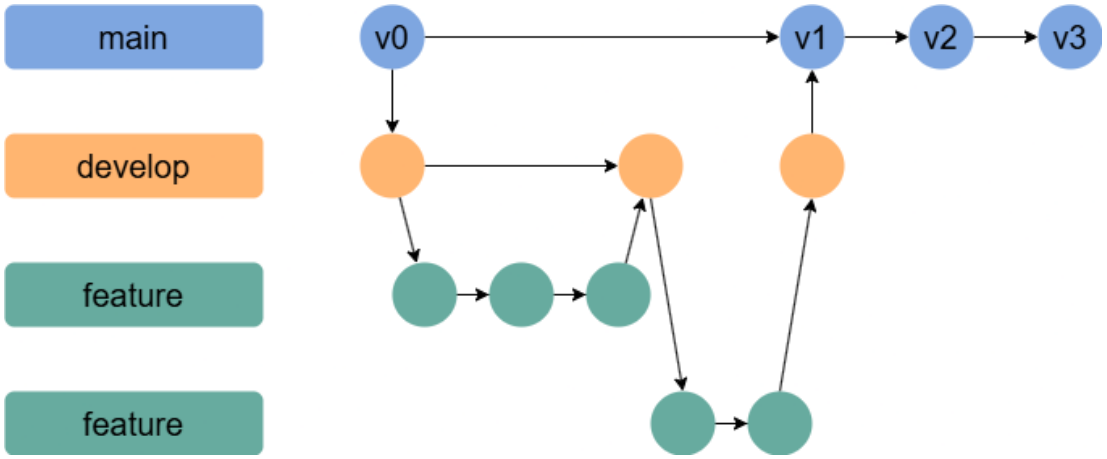
- **À faire** : regroupe les tâches en attente de traitement
- **En cours** : contient les tâches actuellement en cours de développement
- **Terminé** : liste les tâches finalisées et validées

La collaboration se faisait sur **GitLab** pour l'API et sur **GitHub** pour l'interface, des outils qui servent de plateforme centralisée pour la gestion du code.

Pour chaque nouveau ticket, je devais créer une branche spécifique dédiée à la tâche. Par exemple, pour le ticket BP-1707, qui concernait l'ajout d'une nouvelle fonctionnalité, j'ai nommé la branche *feat/BP-1707*.

À chaque modification apportée, je réalisais des commits sur cette branche et lorsque les changements apportés répondaient aux critères du ticket, je créais une merge request vers la branche develop, destinée à être revue par le responsable du ticket.

Une fois que toutes les corrections étaient apportées et validées, la branche était fusionnée vers la branche develop, puis plus tard vers la branche main.



5. Architecture

Pour le backend, j'ai opté pour une architecture **RESTful** basée sur le framework **Spring Boot**, choisi pour sa robustesse et sa flexibilité, permettant de créer des applications web évolutives et maintenables. L'utilisation de **Docker** pour la conteneurisation de l'application a également été un choix stratégique, garantissant une exécution cohérente dans différents environnements.

Pour le frontend, j'ai choisi une architecture basée sur le framework **Angular 18**, qui permet de créer des applications web dynamiques et réactives. Angular offre une structure modulaire et une séparation claire des responsabilités, facilitant ainsi la maintenance et l'évolution de l'application. L'utilisation d'**Angular Material** a permis d'assurer une cohérence visuelle et une expérience utilisateur optimale, tout en respectant les principes du **Material Design**.

6. Développement Backend (API)

6.1. Choix des outils

- **Visual Studio Code :** Éditeur de code utilisé pour le développement de l'API, choisi pour sa légèreté, sa polyvalence et son large écosystème d'extensions.
- **Spring Boot :** Framework Java pour le développement backend, choisi pour sa simplicité d'utilisation et sa compatibilité avec les bases de données relationnelles comme PostgreSQL.
- **Gradle :** Outil de gestion de dépendances et de compilation utilisé pour le projet Spring Boot, choisi pour sa flexibilité et sa compatibilité avec les projets Java modernes.

- **PostgreSQL :** Base de données relationnelle utilisée pour stocker les données des codes postaux et des boîtes postales. Son choix s'est imposé en raison de sa robustesse, mais également parce qu'il s'agit de la base de données déjà utilisée par l'application **SIBP**, ce qui a facilité l'intégration et la reprise des données existantes.
- **Docker :** Plateforme de conteneurisation utilisée pour isoler l'application et ses dépendances, facilitant ainsi le déploiement et la gestion des environnements.
- **Swagger :** Outil de documentation automatique des API REST, choisi pour sa simplicité d'intégration et sa capacité à générer une interface utilisateur interactive.

6.2. Alive

Le premier endpoint que j'ai développé était le endpoint **/alive**, permettant de vérifier l'état de santé de l'application. L'objectif de cet endpoint était de renvoyer un statut indiquant si l'application est opérationnelle, accompagné de la date du jour.

```
{
    "status": "alive",
    "timestamp": "2024-11-14T10:52:02.8064044"
}
```

6.3. Recherche par localité

6.3.1. Version 1

Dans un premier temps, j'ai étudié la structure de la base de données **PostgreSQL** de l'application **SIBP**. Ce travail m'a permis de prendre connaissance de la structure des données ainsi que du rôle spécifique de chaque table et de chaque colonne. Cela m'a notamment aidé à identifier et extraire les tables essentielles pour le projet : **agence**, **code_postal** et **produit**.

J'ai ensuite commencé par développer l'endpoint **/v1/code-postal** qui prend pour paramètre **nomCommune**, dont l'objectif était de permettre la recherche du code postal d'une commune en fonction de son nom.

```
[
    {
        "nomCommune": "NOUMEA",
        "codePostal": 98800
    }
]
```

Si la commune n'est pas trouvée dans la base de données, l'API renvoie une erreur 404, indiquant qu'aucun code postal ne correspond à la commune passée en paramètre.

```
{
    "statusCode": "NOT_FOUND",
    "message": "Aucun code postal ne correspond à cette commune."
}
```

Durant le développement de cet endpoint, j'ai eu l'occasion d'approfondir ma compréhension des tests unitaires et des tests d'intégration. J'ai appris que les tests unitaires se concentrent sur la validation du bon fonctionnement d'une unité de code

isolée, généralement une méthode ou une fonction, tandis que les tests d'intégration vérifient l'interaction correcte entre différentes parties du système.

6.3.2. Version 2

Dans cette nouvelle version, la première amélioration que j'ai apportée a été l'implémentation du pattern **Specification**.

J'ai implémenté ce pattern pour permettre des recherches dynamiques et flexibles sur les codes postaux. Ce pattern repose sur l'utilisation de prédicats pour filtrer les résultats en fonction des critères spécifiés par l'utilisateur. La recherche insensible à la casse des noms de communes a été rendue possible grâce à ce pattern.

Par exemple, en envoyant une requête à l'endpoint **/v2/code-postaux/search?query=mont**, il est désormais possible d'obtenir une réponse listant toutes les communes dont le nom commence par "mont". Cette fonctionnalité enrichit les capacités de recherche de l'API.


```
[
    {
        "nomCommune": "MONT DORE",
        "codePostal": 98809
    },
    {
        "nomCommune": "MONT DORE",
        "codePostal": 98810
    }
]
```


J'ai aussi ajouté un nouvel endpoint **/v2/codes-postaux/communes** pour obtenir la liste des communes dans le but d'ajouter un menu d'auto-complétion dans le front-end.

6.3.3. Version 3

Dans cette troisième version, plusieurs améliorations ont été apportées pour optimiser l'application.

Pour optimiser les requêtes et structurer les données, j'ai créé des vues PostgreSQL telles que **codes_postaux_view** et **boites_postales_view**. Ces vues permettent de regrouper les données nécessaires à chaque endpoint, simplifiant ainsi les requêtes et améliorant les performances.

La recherche a également été rendue plus précise. Jusqu'à présent, elle se basait uniquement sur le champ **localite** de la table **code_postal**, ce qui pouvait entraîner des résultats incomplets. Par exemple, une recherche sur la commune du Mont-Dore n'affichait pas les codes postaux des localités spécifiques telles que Plum ou La Coulée.

Pour résoudre ce problème, une table de correspondance a été créée afin d'établir un lien entre chaque localité et sa commune. Cette association a ensuite été intégrée à la vue **codes_postaux_view**, permettant d'inclure toutes les localités liées à une commune dans les résultats de recherche. Grâce à cela, l'endpoint **/v3/codes-postaux/localites** (anciennement **/communes**) renvoie désormais à la fois les localités et les communes, simplifiant encore davantage le fonctionnement de l'auto-complétion.

Désormais, une recherche affiche correctement toutes ses localités, en distinguant les types de codes postaux : **domiciles**, **boîtes postales**, ou **les deux**.

Pour les codes postaux dédiés aux BPs, leur intervalle est également précisé dans les résultats.

Cette fonctionnalité a été particulièrement complexe à coder, car certains numéros de boîte postale sont alphanumériques, rendant le calcul des intervalles de numéros non continus bien plus difficile.

Dans un premier temps, j'avais envisagé de définir un intervalle en prenant le plus petit et le plus grand numéro de boîte postale, mais en réalité, plusieurs intervalles distincts existent. J'ai donc ensuite essayé d'exploiter les valeurs ASCII, puis les valeurs hexadécimales de chaque caractère, mais aucune de ces approches ne me donnait des intervalles correctes.

Finalement, avec l'aide de David, un développeur du SI Postal, nous avons conçu un algorithme simple permettant d'identifier correctement tous les intervalles distincts.

Grâce à cette solution, on obtient désormais un résultat comme celui présenté ci-dessous :

```
[
    {
        "id": 3850,
        "commune": "MONT DORE",
        "localite": "PONT DES FRANÇAIS",
        "code": 98874,
        "typeDomicile": false,
        "typeBp": true,
        "intervalles": [
            [
                "1001",
                "2700"
            ]
        ]
    },
    // ... autres réponses ...
]
```

6.4. Recherche par numéro de boîte postale

6.4.1. Version 1

Après validation du premier endpoint, j'ai pu passer au développement du second, à savoir **/v1/boite-postale** qui prend pour paramètre **numeroBP**. L'objectif de cet endpoint était de permettre la recherche des codes postaux, des agences et des localités associés à un numéro de boîte postale donné. Si la boîte postale est trouvée, l'API renvoie une réponse au format **JSON** contenant les informations.

Un ajout significatif à cet endpoint a été l'implémentation de la pagination, afin de limiter le nombre de résultats retournés à un certain nombre d'éléments à la fois. Cette pagination est contrôlée via un paramètre **pageNo** pour le numéro de page et **pageSize** pour la taille de chacune des pages.

```
{
    "content": [
        {
            "nomLocalite": "NOUMEA CEDEX",
            "codePostal": 98802,
            "nomAgence": "AGENCE DE NOUMEA MAGENTA"
        }
    ],
    "pageable": {
        "pageNumber": 0,
        "pageSize": 10,
        "sort": {
            "empty": true,
            "sorted": false,
            "unsorted": true
        },
        "offset": 0,
        "paged": true,
        "unpaged": false
    },
    "last": true,
    "totalElements": 1,
    "totalPages": 1,
    "first": true,
    "numberOfElements": 1,
    "size": 10,
    "number": 0,
    "sort": {
        "empty": true,
        "sorted": false,
        "unsorted": true
    },
    "empty": false
}
```


En revanche, si la boîte postale n'est pas trouvée, une réponse d'erreur 404 est renvoyée, accompagnée d'un message d'erreur clair et approprié.


```
{
    "statusCode": "NOT_FOUND",
    "message": "Aucune boite postale ne correspond à ce numéro."
}
```


6.4.2. Version 3

De la même façon que pour la recherche par localité, j'ai créé une vue nommée **boites_postales_view** afin de regrouper toutes les informations nécessaires de manière plus structurée et accessible. L'endpoint a également été renommé en **v3/boites-postales/search**, pour suivre la même nomenclature que la recherche des codes postaux.

Une autre amélioration a été l'ajout de la prise en charge des numéros au format alphanumérique. Auparavant, seuls les numéros de boîtes postales strictement numériques étaient acceptés, ce qui excluait certaines boîtes postales, notamment celles attribuées à des entreprises.

Ce qui donne, par exemple, la réponse suivante :

```
{
    "content": [
        {
            "id": 15268,
            "agence": "AGENCE DE NOUMEA MAGENTA",
            "numero": "MGA40",
            "codePostal": 98802,
            "localite": "NOUMEA CEDEX"
        }
    ],
    // ... paramètres de pagination …
}
```

6.5. Swagger

J'ai implémenté **Swagger**, un outil permettant de documenter automatiquement les API REST. Swagger s'appuie sur les annotations présentes dans le code pour générer une documentation interactive, accessible à l'adresse : localhost:8080/swagger-ui/index.html

6.6. Conteneurisation

Pour faciliter le déploiement de l'application dans différents environnements, j'ai utilisé **Docker**, une plateforme de conteneurisation qui isole l'application avec ses dépendances, garantissant son fonctionnement cohérent. J'ai créé un **Dockerfile** pour l'API Spring Boot, définissant les étapes pour construire une image. De plus, j'ai utilisé **Docker Compose** pour gérer les multiples services, comme PostgreSQL et l'API, permettant de lancer l'application avec une seule commande et assurant la communication entre les services.

7. Développement Frontend (Interface)

7.1. Choix des outils

- **Visual Studio Code :** Éditeur de code utilisé pour le développement du frontend, choisi pour sa légèreté, sa polyvalence et son large écosystème d'extensions.
- **Angular 18 :** Framework JavaScript open-source développé par Google, choisi pour sa robustesse, sa modularité et sa capacité à créer des applications web modernes et réactives.
- **Angular Material :** Librairie de composants UI basée sur les principes du Material Design, utilisée pour garantir une interface utilisateur cohérente et esthétique.
- **Node.js :** Environnement d'exécution JavaScript.
- **NPM :** Gestionnaire de paquets utilisés pour installer et gérer les dépendances du projet
- **Angular CLI :** Outil en ligne de commande pour Angular, utilisé pour initialiser le projet, générer des composants, des services et des modules, ainsi que pour compiler et déployer l'application.
- **GitHub Pages :** Plateforme utilisée pour héberger et déployer l'application frontend.

7.2. Maquettage

Avant de débuter le développement, j'ai conçu des maquettes à l'aide de l'outil Figma pour concrétiser le futur aspect de l'application. Ces maquettes, disponibles en annexe, ont permis de visualiser et de valider l'**interface utilisateur (UI)** et l'**expérience utilisateur (UX)** avec les parties prenantes, en définissant clairement la structure, les fonctionnalités et l'ergonomie de l'application.

7.3. Recherche par localité

7.3.1. Version 1

J'ai commencé par créer un nouveau composant intitulé **codes-postaux** destiné à représenter la racine de la recherche des codes postaux. Pour sa mise en œuvre, j'ai utilisé le module Angular **ReactiveFormModule**.



Un validateur a d'ailleurs été intégré pour empêcher les entrées vides.



Les données saisies dans ce formulaire sont passées en paramètre de l'endpoint **/v1/code-postal** de l'API, afin de récupérer les informations correspondantes.

Une fois la recherche fonctionnelle, je suis passé à l'affichage des données. J'ai créé un second composant **boites-infos** dédié à l'affichage des résultats de recherche sous forme de boîte d'informations, respectant la forme définie dans les maquettes.



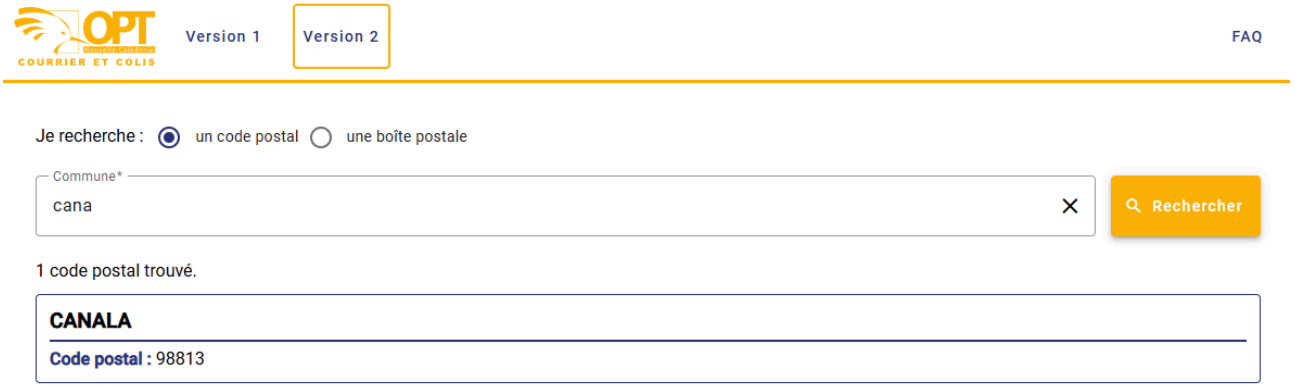
Comme on peut le voir, j'ai créé un thème Angular Material aux couleurs de l'OPT-NC afin d'assurer une cohérence visuelle.

Pour la version 1, cela se traduit par une interface utilisateur simple et fonctionnelle, comme illustré ci-dessous.

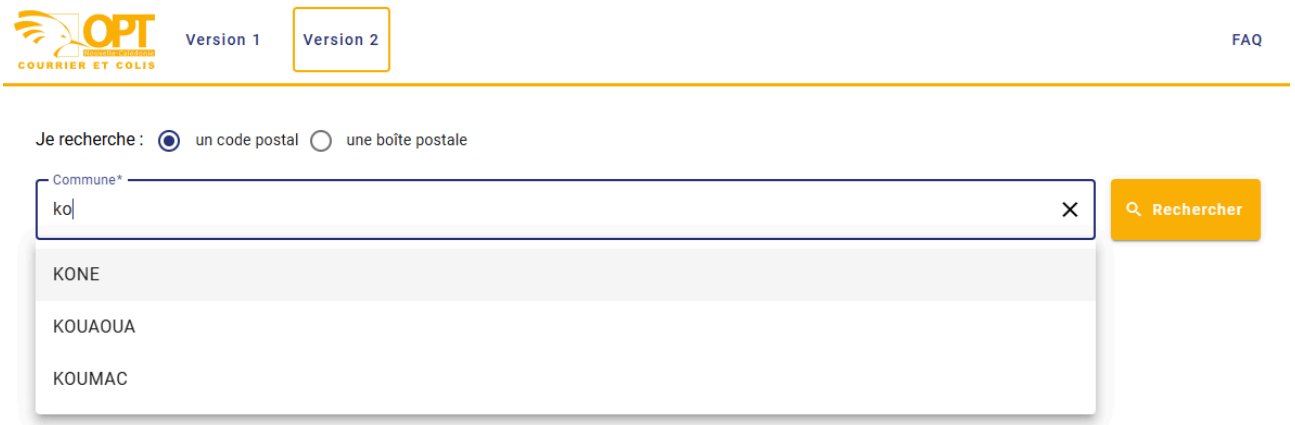


7.3.2. Version 2

J'ai utilisé l'endpoint **/v2/codes-postaux/search**, qui, grâce à l'implémentation du pattern Specification, permet une recherche plus flexible et adaptée aux critères spécifiés par l'utilisateur.



J'ai ensuite intégré le composant **Mat Autocomplete** d'Angular Material pour proposer une liste déroulante des communes disponibles. Cette fonctionnalité utilise l'endpoint **/v2/codes-postaux/communes** de l'API, permettant de simplifier et d'accélérer la saisie pour l'utilisateur.

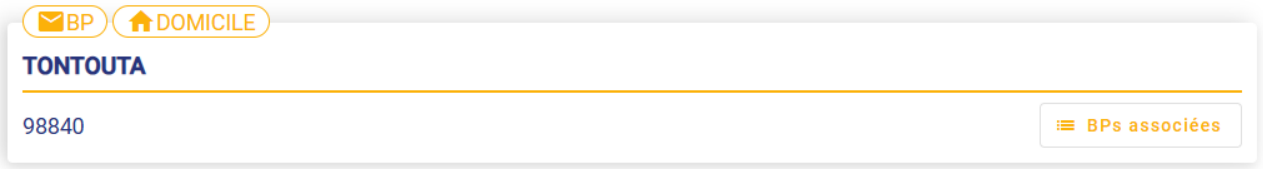


7.3.3. Version 3

Dans cette dernière version, plusieurs améliorations visuelles et techniques ont été apportées pour améliorer l'expérience utilisateur et rendre le code plus propre. Les zones de recherche et de résultats ont été rétrécies et recentrées afin de maintenir les informations essentielles au centre de l'attention.

Les petites cartes utilisées pour présenter les résultats ont également été revisitées pour adopter un style plus moderne.

Une autre amélioration notable concerne l'ajout de l'affichage du type de chaque code postal. Désormais, il est indiqué si un code postal correspond à des boîtes postales, à un domicile, ou aux deux.



Du côté technique, j'ai pris la décision d'abandonner l'utilisation du composant **boites-infos**, créé initialement dans la v1 puisqu'il n'était plus assez modulaire pour les deux types de recherche. Je l'ai donc remplacé par deux composants distincts, dont le nouveau composant **resultat-cp**.

La principale nouveauté de cette version réside dans la possibilité d'afficher les intervalles des boîtes postales associées à un code postal. En cliquant sur le bouton **"BPs associées"**, l'utilisateur peut visualiser ces informations sous forme de tableau.



7.4. Recherche par numéro de boîte postale

7.4.1. Version 1

De la même manière que pour la recherche des codes postaux, j'ai créé un nouveau composant dédié à la recherche par numéro de boîte postale. Ce composant présente une structure similaire, avec quelques différences notables.

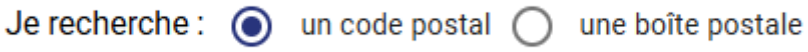
En plus du validateur empêchant les entrées vides, un validateur spécifique a été ajouté pour le champ de saisie, imposant l'utilisation exclusive de chiffres. Cela permet d'éviter des appels inutiles à l'API.



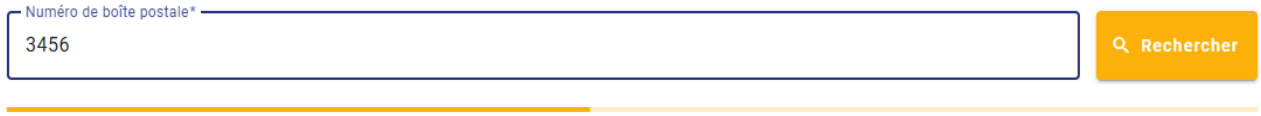
Pour la récupération des informations, l'endpoint **/v1/boite-postale** avec le paramètre **numeroBP** a été utilisé. Cependant, cet endpoint retourne des objets au format **Page**, nécessitant un traitement pour mettre les données en forme. J'ai opté pour le composant **Paginator** d'Angular Material, une solution simple et efficace pour gérer la pagination.



Une fonctionnalité a également été ajoutée pour permettre à l'utilisateur de sélectionner le type de recherche souhaité, que ce soit pour un code postal ou une boîte postale.



Des optimisations ont été réalisées pour éviter les requêtes inutiles à l'API. Si l'utilisateur valide une recherche sans modifier la valeur du champ, aucune nouvelle requête n'est envoyée. Une barre de chargement a été intégrée pour indiquer à l'utilisateur que la recherche est en cours.



Ces améliorations ont également été implémentées dans le formulaire de recherche des codes postaux.

De la même manière que pour la recherche des codes postaux, j'ai utilisé le composant **boites-infos** pour formater la réponse de la recherche.



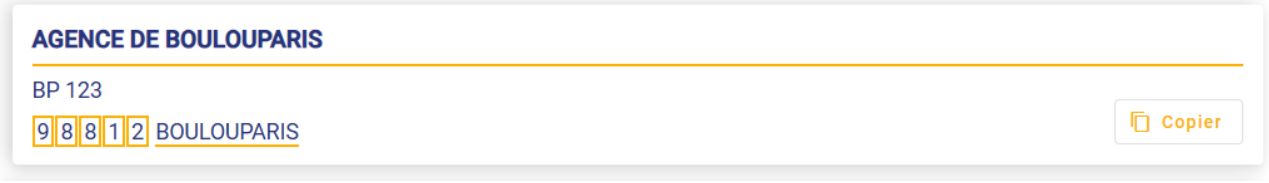
7.4.2. Version 3

Pour finir, cette version a également bénéficié d'améliorations visuelles, similaires à celles apportées à la recherche des codes postaux, afin de donner un style plus moderne à l'interface.

J'ai également remplacé le composant **boites-infos** par un composant dédié, nommé **resultat-bp**.

Le format d'affichage a été ajusté pour mieux correspondre à la présentation attendue sur une enveloppe, facilitant ainsi l'utilisation pour les utilisateurs.

De plus, un bouton utilitaire a été ajouté pour permettre la copie rapide du numéro de boîte postale, du code postal et de la localité.



7.5. Intégration et déploiement continus (CI/CD)

Ici **GitHub** a été utilisé plutôt que **GitLab**, car GitHub permet de déployer directement une application Angular via **GitHub Pages**. Pour automatiser ce déploiement, j'ai mis en place une **GitHub Action** qui gère l'**intégration continue (CI)** et le **déploiement continu (CD)**.

Avec l'intégration continue, à chaque fusion vers la branche develop, la GitHub Action s'exécute automatiquement en effectuant plusieurs étapes, comme l'installation des packages et le build de l'application Angular.

Pour le déploiement continu, une fois que la l'application est prête, la GitHub Action déploie automatiquement la version "buildée" de l'application sur GitHub Pages, assurant ainsi que la version en ligne reflète toujours les dernières modifications de la branche develop.

De cette manière, une démonstration en ligne de l'application est disponible à l'adresse suivante : https://malcolmbrt.github.io/bp-front-code-postaux/v3

8. Ateliers

Au cours de mon stage, j'ai participé à divers ateliers visant à discuter du produit et de ses objectifs. Pour chaque atelier, je rédigeais un compte rendu sur Confluence, l'espace de documentation utilisé par l'équipe. Ces comptes rendus détaillent les sujets abordés ainsi que les actions à mener par la suite.

8.1. Définir les cas d'utilisation de l'API

Le premier atelier, qui s'est déroulé le 6 novembre, avait pour objectif de définir les différents cas d'utilisation de l'API.

Plus précisément, j'ai été chargé de créer des schémas détaillés pour chaque cas d'utilisation identifié lors de l'atelier. Ces schémas ont servi de base pour formaliser les besoins fonctionnels et techniques de l'API, en mettant en avant des scénarios tels que la

recherche d’un code postal pour une commune ou la récupération d’informations liées à une boîte postale (agence, code postal, localité).

Du côté de mes encadrants, leur tâche consistait à créer des tickets sur Jira pour répondre aux besoins identifiés lors de cet atelier.

En résumé, à l’issue de cette réunion, mes principales tâches ont été de finaliser les schémas des cas d’utilisation et de m’assurer qu’ils étaient suffisamment détaillés pour guider les développements ultérieurs.

8.2. Présentation de mon travail auprès du métier

Lors de ce second atelier, qui s’est tenu le 9 décembre, j’ai présenté l’ensemble des travaux réalisés jusqu’à présent, notamment les avancées jusqu’à la version 2 de l’application. Cette présentation a permis d’échanger avec les équipes métier afin de recueillir leurs retours et de déterminer les améliorations nécessaires pour la version 3, qui constitue la version finale du projet.

Au cours de cet atelier, les discussions ont porté sur plusieurs points d’amélioration, notamment :

1. Recherche par localité :

- Il a été demandé de préciser le type de codes postaux (BP, domicile ou les deux).
- Pour les codes postaux attribués à des boîtes postales, il a été suggéré de préciser l’intervalle correspondant.
- Une demande a été faite pour élargir la recherche aux communes et localités associées (par exemple, une recherche pour "Mont Dore" devrait retourner les codes postaux pour Mont Dore ainsi que pour Plum, Pont des Français, La Coulée, etc.).

2. Recherche par boîte postale :

- Il a été demandé de modifier le format de réponse pour qu’il corresponde mieux à ce que l’expéditeur doit écrire sur son enveloppe.
- Il a été suggéré d’ajuster le format d’acceptation des numéros de boîtes postales pour accepter des valeurs alphanumériques jusqu’à 5 caractères maximum.
- Une demande a été faite pour rendre l’application responsive, afin qu’elle s’adapte correctement à différents appareils (ordinateurs, tablettes, smartphones).
- Il a également été proposé d’implémenter une fonctionnalité d’auto-focus sur le formulaire dès le rafraîchissement de la page.

En résumé, cet atelier a été l’occasion de présenter les avancées techniques et de recueillir des retours constructifs pour finaliser l’application.

8.3. Présentation finale auprès du métier

À la suite des retours recueillis lors du précédent atelier, j'ai pu mettre en œuvre les améliorations et modifications nécessaires. Dans ce cadre, j’ai eu l’opportunité d’assister à un dernier atelier en visio le 30 décembre, juste avant la fin de mon stage, pour présenter la version finale de mon projet. Cette présentation finale m’a permis de montrer les ajustements réalisés.

9. Problèmes rencontrés

9.1. Difficulté à trouver un stage

Les émeutes et les destructions en Nouvelle-Calédonie ont rendu la recherche de stage particulièrement difficile, car les entreprises étaient réticentes à recruter des stagiaires. J'ai donc multiplié les candidatures spontanées et les relances, ce qui m'a permis de décrocher un stage à l'OPT-NC en septembre, aligné avec mes compétences et mes intérêts.

9.2. Compréhension de la base de données SIBP

L'application devait se baser sur la base de données peu documentée de SIBP. Pour y faire face, j'ai pris le temps d'étudier la structure des tables et leurs relations, ce qui m'a permis de concevoir des vues PostgreSQL optimisant les requêtes.

9.3. Gestion des recherches par code postal

Initialement, l'API ne retournait pas toutes les localités liées à une commune, ce qui rendait les résultats incomplets. Pour résoudre ce problème, j'ai créé une table de correspondance entre localités et communes, que j'ai ensuite intégrée dans une vue PostgreSQL pour affiner la précision des recherches.

9.4. Gestion des intervalles de boîtes postales

Certains numéros de boîtes postales étaient alphanumériques et non continus, ce qui compliquait la gestion des intervalles. Après plusieurs tentatives infructueuses (conversion ASCII, hexadécimal), j'ai collaboré avec un développeur du SI Postal pour créer un algorithme permettant de détecter correctement les intervalles distincts.

10. Conclusion

Ce stage m'a offert une opportunité unique de travailler sur un produit innovant et utile, tout en endossant successivement les rôles de développeur backend, puis frontend. Grâce à cette expérience, j'ai pu appréhender les deux facettes du développement web, tout comme un développeur full-stack.

La confiance accordée par mes encadrants m'a permis de gagner en autonomie tout au long du projet, jusqu'à la rédaction, gestion et validation de mes propres tickets. Cela m'a non seulement permis d'acquérir des compétences techniques précieuses, mais également de me familiariser avec les responsabilités et l'organisation d'un projet.

Réaliser ce stage au sein d'une grande entreprise telle que l'OPT-NC a également été une expérience particulièrement enrichissante. J'ai découvert la réalité du monde professionnel, ses exigences, et son organisation.

Ce stage a été un véritable levier pour l'acquisition de nouvelles compétences techniques qui me seront indispensables dans mon parcours professionnel. En effet, les outils et frameworks que j'ai appris à maîtriser, tels qu'Angular et Spring Boot, sont largement utilisés dans le milieu professionnel, renforçant ainsi mon employabilité et ma préparation pour mes futures expériences.

Enfin, cette expérience m'a confortée dans mon attrait pour le développement web, tout en éveillant ma curiosité pour d'autres domaines de l'informatique. Les multiples facettes de ce secteur me motivent à poursuivre mes études supérieures, afin d'explorer de nouveaux horizons.

11. Glossaire

API : Interface logicielle qui permet de « connecter » un logiciel ou un service à un autre logiciel ou service afin d'échanger des données.

Angular : Framework JavaScript open-source développé par Google, utilisé pour construire des applications web dynamiques et réactives.

Backend : Correspond à la partie d'une application ou d'un site "coté serveur", c'est-à-dire celle que ne peuvent pas voir les utilisateurs. C'est donc un applicatif qui tourne sur le serveur de l'application mobile ou web.

Composants web : Éléments modulaires et réutilisables qui constituent l'interface utilisateur d'une application web.

Docker : Plateforme open source qui permet aux développeurs de créer, déployer, exécuter, mettre à jour et gérer des applications conteneurisées.

Endpoint : Point de connexion d'une API, représentant une URL spécifique où les clients peuvent envoyer des requêtes pour interagir avec le serveur.

Framework : Ensemble d'outils, de standards et de composants logiciels accélérant et facilitant le développement d'une application.

Frontend : Correspond à l'interface utilisateur à proprement dit. C'est celle qui sera vue par les utilisateurs et à travers laquelle ils pourront naviguer.

GitLab / GitHub : Plateformes open source de gestion de versions et de collaboration destinée aux développeurs de logiciels. Elles reposent sur Git, un système de gestion de code également open source.

Intégration Continue / Déploiement Continu (CI / CD) : Pratiques de développement logiciel qui consistent à automatiser les processus de build, de test et de déploiement pour accélérer le cycle de développement et réduire les erreurs.

JSON : Format de données textuel léger dérivé de la notation des objets du langage JavaScript.

Material Design : Système de conception développé par Google qui définit des principes et des composants pour créer des interfaces utilisateur cohérentes et esthétiques.

Packages : Collections de fichiers ou bibliothèques prêtes à être utilisées dans un projet, permettant de réutiliser du code existant pour gagner du temps.

Pagination : Technique utilisée en développement pour diviser les données d'une API ou d'une application en plusieurs pages, facilitant ainsi leur affichage et leur gestion.

Pattern : Solution réutilisable pour résoudre des problèmes récurrents en développement logiciel.

PostgreSQL : Système de gestion de base de données relationnelle orienté objet connu pour sa robustesse et sa capacité à gérer des requêtes complexes.

REST : Style d'architecture logicielle définissant un ensemble de contraintes à utiliser pour créer des services web.

Spring Boot : Outil qui simplifie le développement d'applications Web et de microservices avec Spring Framework.

Stack : Ensemble de technologies utilisées pour développer une application.

Swagger : Outil permettant de documenter et de tester facilement les API REST à travers une interface utilisateur interactive.

Système d'Information (SI) : Ensemble de composants, de technologies et d'acteurs interconnectés pour gérer les données et l'information dans une organisation.

Expérience Utilisateur (UX) : Manière dont une personne perçoit et interagit avec un produit ou un service numérique.

Validateur : Mécanisme ou fonction utilisée pour vérifier qu'une donnée saisie par un utilisateur ou une requête respecte des règles définies.

Vue : Vue logique dans une base de données permettant de regrouper et structurer des données issues de plusieurs tables, facilitant les requêtes complexes.

Workflow : Série d'étapes, de processus ou de tâches organisés de manière séquentielle pour atteindre un objectif spécifique.

12. Références

OPT-NC, 2020. Tableau des codes postaux [en ligne]. Disponible sur : https://www.opt.nc/particuliers/courrier-et-colis/les-codes-postaux-de-nouvelle-caledonie [consulté le 28 janvier 2025].

OPT-NC, 2023. Portail des Services Postaux [en ligne]. Disponible sur : https://monespace.opt.nc [consulté le 28 janvier 2025].

Angular, 2023. Documentation officielle [en ligne]. Disponible sur : https://angular.dev/overview [consulté le 28 janvier 2025].

Spring Boot, 2014. Documentation officielle [en ligne]. Disponible sur : https://spring.io/projects/spring-boot [consulté le 28 janvier 2025].

Swagger UI, 2015. Documentation de l'interface utilisateur [en ligne]. Disponible sur : https://swagger.io/tools/swagger-ui [consulté le 28 janvier 2025].

PostgreSQL, 2021. Documentation officielle sur les vues [en ligne]. Disponible sur : https://www.postgresql.org/docs/current/tutorial-views.html [consulté le 28 janvier 2025].

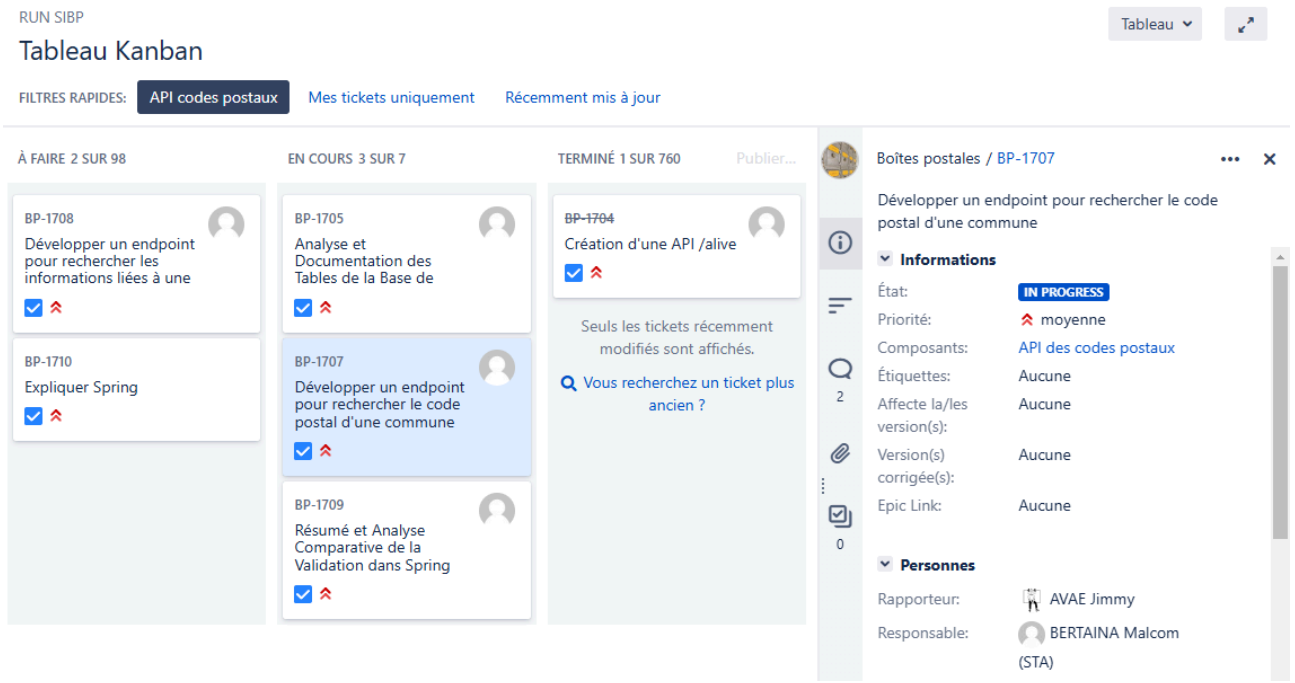
GitHub Actions, 2022. Documentation officielle [en ligne]. Disponible sur : https://docs.github.com/fr/actions [consulté le 28 janvier 2025].

Material Design, 2021. Principes et documentation [en ligne]. Disponible sur : https://m3.material.io [consulté le 28 janvier 2025].

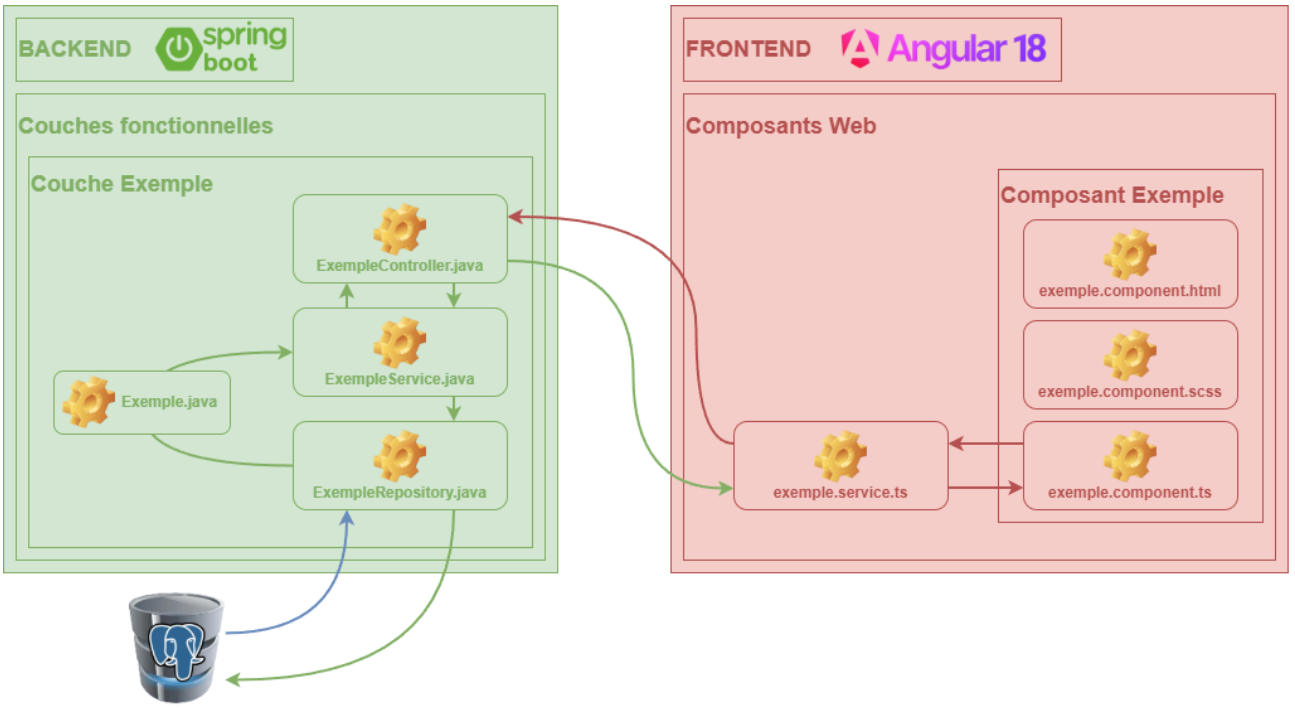
Jira, 2014. Documentation officielle sur les tableaux Kanban [en ligne]. Disponible sur : https://www.atlassian.com/agile/kanban [consulté le 28 janvier 2025].

13. Annexes

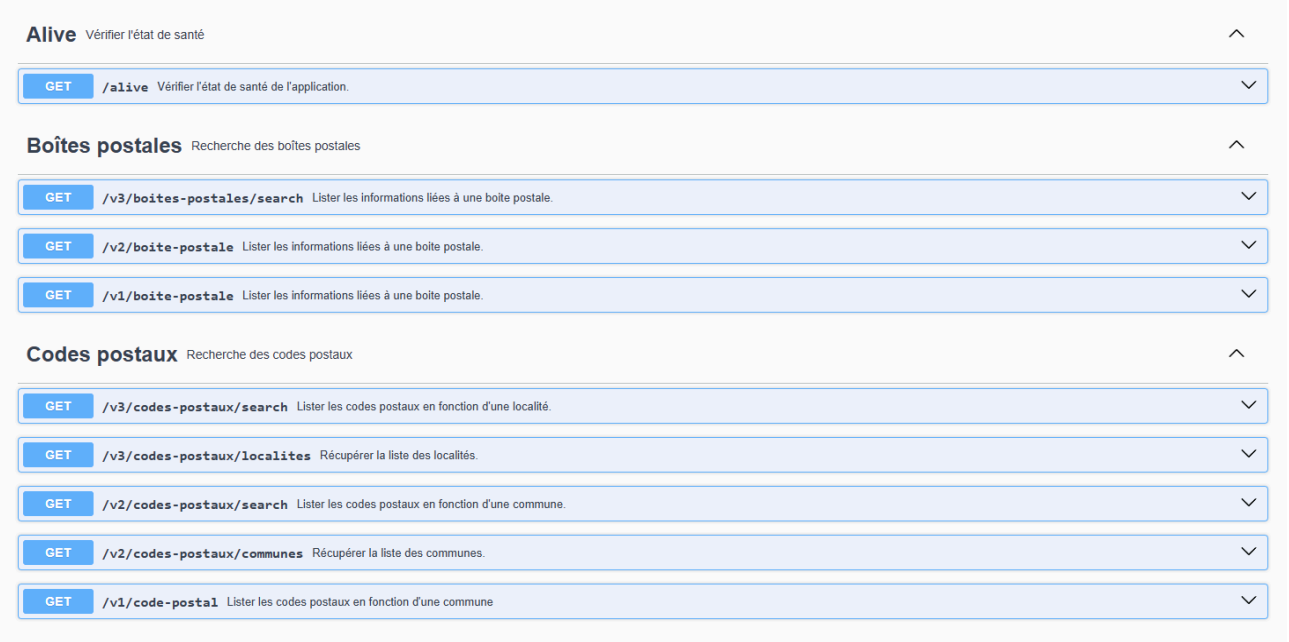
Annexe 1 : Tableau Kanban sur Jira



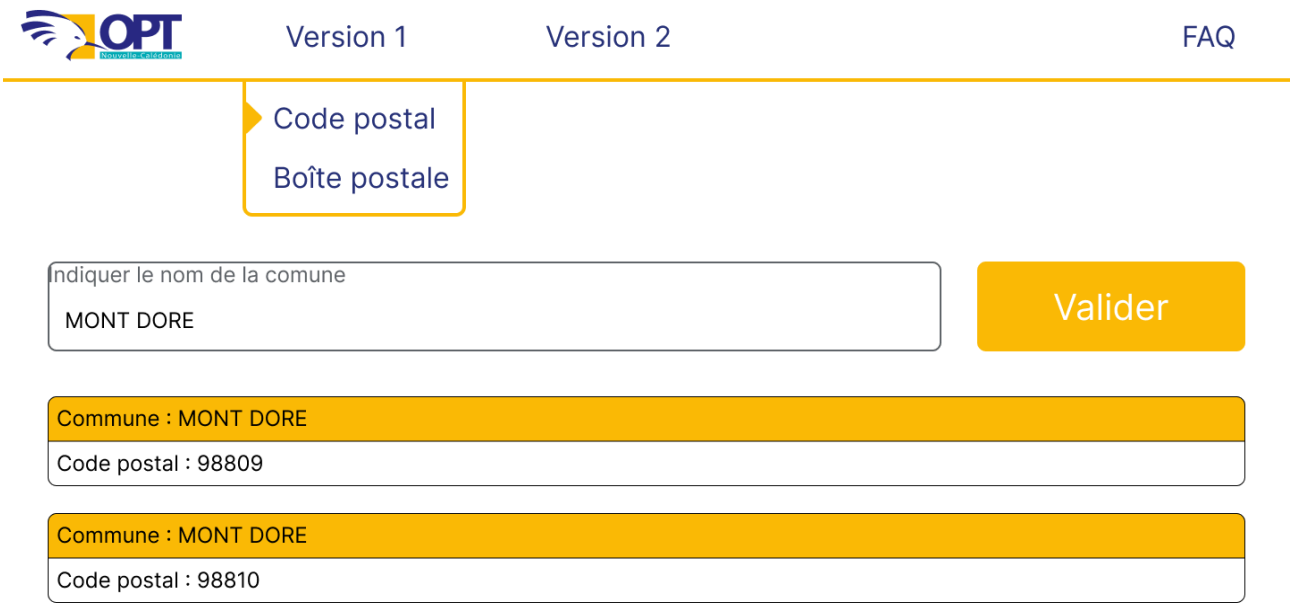
Annexe 2 : Schéma global de l'architecture logiciel



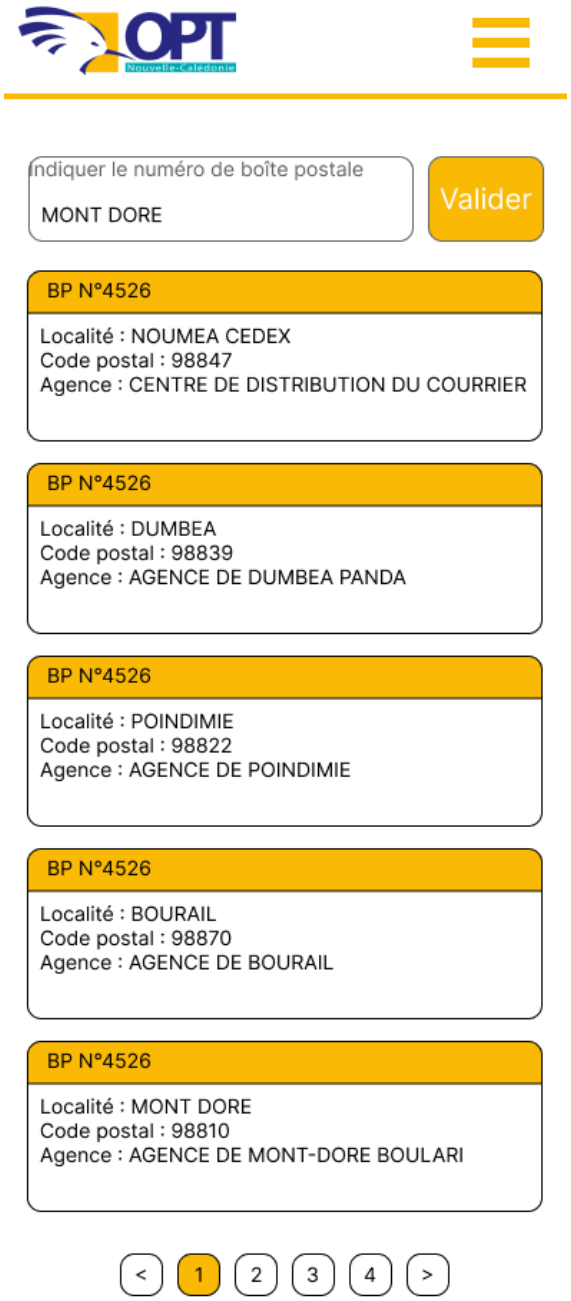
Annexe 3 : Interface de documentation Swagger



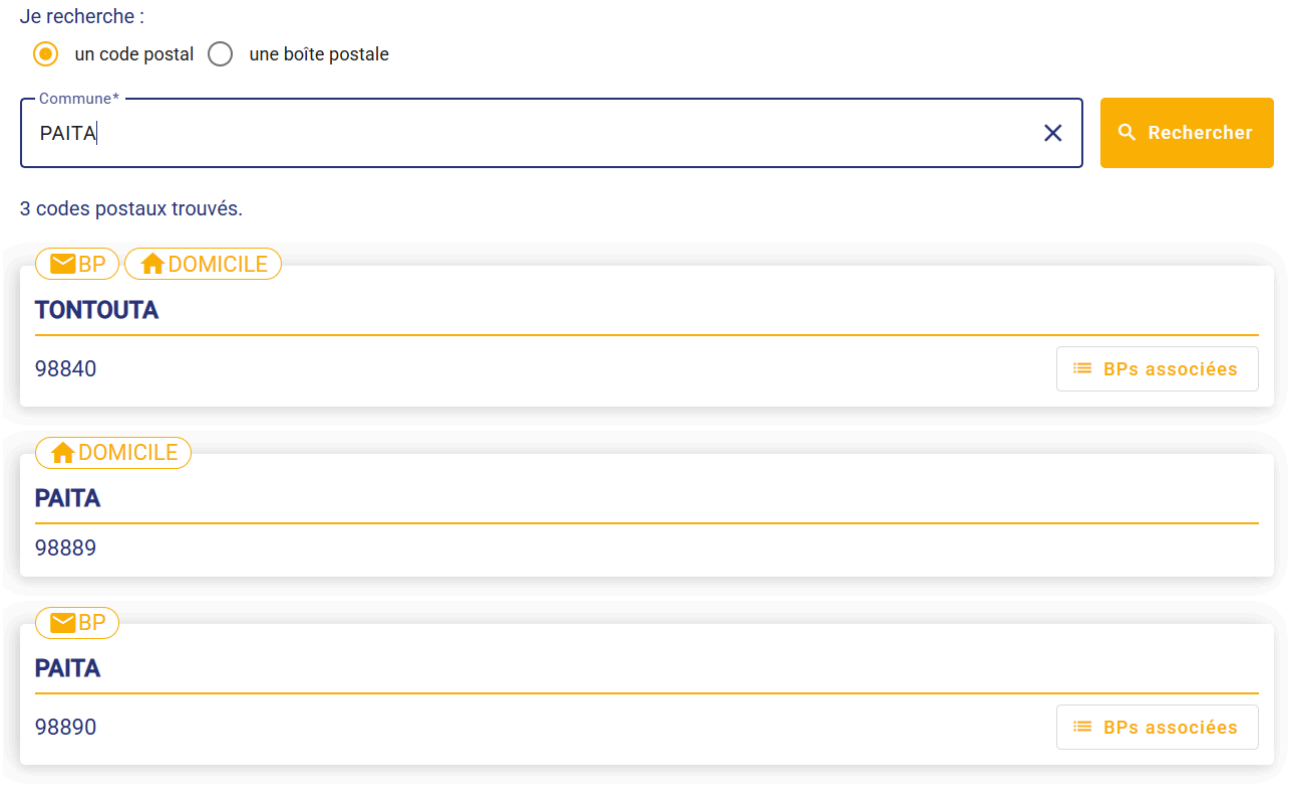
Annexe 4 : Maquette recherche par localité



Annexe 5 : Maquette recherche par numéro de boîte postale



Annexe 6 : Interface final de la recherche par localité



Annexe 7 : Interface final de la recherche par numéro de boîte postale

